CONTACT MANAGEMENT SYSTEM...

Problem Statement :

Develop a contact management system using a doubly linked list, an array and a stack. Contact details (ID, name, phone, slot number) are stored in the list, contact slots in the array, and deleted contacts in the stack so the last delete can be undone. It supports add, delete, search, and forward and backward display.

Objective :

The objective of this project is to develop a contact management system in C++ using structures without classes. Contact details such as ID, name, phone number and allotted slot number are stored in a doubly linked list which allows easy insertion, deletion, searching and traversal in both forward and backward direction. Contact slots are managed using an array so the system can allot a free slot to each new contact. Deleted contacts are stored in a stack which keeps the delete history and lets the last delete be undone. Through this program we apply linked list, stack and array concepts to solve a real-world contact problem efficiently.

Algorithm :-

Step 1: Start. Create an empty doubly linked list, an array of 10 free slots and an empty stack.

Step 2: Display the menu: add, delete, search, display forward, display backward, slot status, undo delete, delete history, exit.

Step 3: Read the choice and call the matching function.

Step 4: Repeat Steps 2 and 3 until choice = 9.

Step 5: Find the first free slot in the array. If none is free display "No slots available" and stop.

Step 6: Read contact ID. If the ID already exists in the list stop.

Step 7: Read name and phone number.

Step 8: Create a new node and insert it at the end of the list (update tail and prev/next links).

Step 9: Mark the slot as occupied and increase count.

Step 10: Read contact ID and search the list from head. If not found display "Contact not found" and stop.

Step 11: Push the contact record into the stack (top = top + 1).

Step 12: Free the contact slot in the array.

Step 13: Delete the node by linking its previous and next nodes (update head or tail if needed).

Step 14: Decrease count.

Step 15: Read the contact ID.

Step 16: Traverse the list from head using next until NULL.

Step 17: If a node matches display the contact details otherwise display "Contact not found".

Step 18: Forward: start at head and print each node using next until NULL.

Step 19: Backward: start at tail and print each node using prev until NULL.

Step 20: If the stack is empty or no slot is free, display a message and stop.

Step 21: Pop the top record (top = top - 1).

Step 22: Insert the contact at the end of the list and mark a free slot as occupied.

Step 23: Slot status: traverse the slot array and print each slot as Free or Occupied.

Step 24: History: print the stack records from top to 0 (latest delete first).

Step 25: Stop.